

Cloud Architecture Anti-Patterns (and What to Do Instead)

Seventeen patterns I flag again and again in cloud architecture reviews — glue-code functions, synchronous long jobs, polling, credentials on disk, jump hosts — each with the alternative I recommend.

September 26, 2026 · 4 min read

<https://brianpfeil.com/architecture/architecture-anti-patterns/>

These are the patterns I flag most often in architecture reviews. None of them is a disaster on its own. Each one adds operational work, hides a failure mode, or makes the next change harder — and they compound.

The examples use AWS service names because that's what I review most, but every major cloud has an equivalent.

Integration

Glue code in a function

Instead: a direct service integration. If a function exists only to take a message from one managed service and hand it to another, let the services talk directly — API Gateway straight to a queue, a workflow calling an SDK integration, an event rule targeting the next service. **Code is a liability:** every line has to be tested, patched and owned.

One big function or script that does many steps

Instead: a workflow engine (Step Functions), one step per function. You get retries, error handling, timeouts and per-step visibility for free, and each step stays small enough to reason about.

Several workflows chained together

Instead: one parent workflow that calls the others. A run is then one thing you can see, retry and reason about, rather than a relay race.

A synchronous API in front of a long job

Instead: accept the request onto a queue and return immediately with an ID; let the client poll (or be notified) for the result. Synchronous calls turn slow work into timeouts and retries into duplicate work.

Storage events wired straight to a function

Instead: put a queue in between — *stable storage first*. The queue absorbs bursts, retries failures and gives you a dead-letter queue to inspect, instead of silently dropping events when the function is throttled.

Calling a flaky or someone-else's downstream API directly

Instead: go through a queue or event bus, so you get buffering and back-pressure. Their outage becomes your backlog, not your outage.

Polling and cron

Instead: event-driven triggers. A job that wakes every five minutes to ask "anything new?" is either late or wasteful, and it's usually the only reason the design needs a server. Most sources can emit an event when something changes.

Compute

A fixed fleet of virtual machines

Instead: managed or serverless compute, or at least an auto-scaling group. See [the compute ladder](#).

Routing logic in application code

Instead: let the API gateway or load balancer route. Routing in configuration is visible, reviewable and changeable without a deploy.

Function URLs for internal APIs

Instead: an API gateway with a private endpoint, so internal APIs get authentication, throttling, logging and a stable name in one place.

Security

Credentials in environment variables or on disk

Instead: a secrets manager or parameter store, read at runtime by a role that's allowed to read exactly that secret — with rotation turned on.

Automation running on a person's credentials, or long-lived keys

Instead: roles. Pipelines, scheduled jobs and integrations should assume a role with temporary credentials. When the person leaves, nothing breaks; when a key leaks, it has already expired.

SSH through a jump host

Instead: a session manager (Systems Manager Session Manager or similar): no inbound ports, no shared keys, and every session is logged.

Data

A relational database by default

Instead: ask what the access pattern really is. Analytics over files is often object storage plus a SQL query layer; spiky transactional work may suit a serverless relational database or a key-value store. An always-on database for a workload that runs twice a day is paying for a lot of idle.

Raw CSV as the long-term format

Instead: a columnar format (Parquet) with lifecycle rules that move old data to cheaper tiers. Queries get faster and cheaper, and retention stops being manual.

Database links between systems

Instead: an API or events. A database link couples two systems' schemas, release schedules and outages, and hides the dependency from everyone who reads the architecture diagram.

Tooling

Two tools doing one job

Instead: pick one. Two API management layers, or a CI server moonlighting as the integration runtime, doubles the patching, the permissions and the places to look when something breaks.

The common thread: **prefer managed services composed through configuration, connected by queues and events, and accessed through roles.** Most of the anti-patterns above are a missing queue, a missing role, or code standing in for a service that already exists.

Want the questions that surface these? They're in [the architecture review checklist](#).